COMP

411/511

Programming Assignment 1: Parsing and Abstract Syntax

Due: 10:59 am Friday, January 19, 2024

Points: 100

Overview

Preparation

We recommend using either IntelliJ or DrJava as your Java IDE for all programming assignments. You can download DrJava in the form of the executable jar file drjava.jar from http://www.cs.rice.edu/~javaplt/drjavarice. DrJava only runs using Java 8. It should run on either Amazon Corretto 8 or Oracle JDK8. If you use IntelliJ as your IDE, you can choose any compatible Java 8 JDK. Beware of the fact that the assignments you submit must support the same API as the starter repository. The easy way to satisfy this property is to simply edit the files in the starter repository, filling in the elided code marked by **...** in the stub files. You are free to add any additional classes that you choose. If you add JUnit test classes, make sure that the class name ends in "**Test**" and that test methods begin with the prefix "**test**", which are the conventions (from JUnit 3.8) followed in DrJava . If your program modifies the API, you**r** program will fail most if not all of the grading correctness tests. You can confirm that your program supports the specified API by running the sample test suite (with all commented-out tests commented back in) in **Assign1Test.java**. Edit the **README** file for your repository to give an overview of your solution. We recommend that you place your Java source files in the folder named src that is already present in your repository.

DrJava is distributed as a Java **jar** file at the local web site

(http://www.cs.rice.edu/~javaplt/drjavarice). DrJava is also available from **sourceforge.net**, but the local site typically has more recent builds. You can run DrJava on a Linux or Mac system (with Java JDK 8 installed) by typing

java -jar drjava.jar

assuming that **drjava.jar** is located in the current directory. Mac OS X can be difficult to configure to run DrJava; if you have problem consult Barret Glass who has DrJava running on his Mac.

On Windows machines (with a properly installed Java JDK 8), you can run the **drjava.jar** file simply by double-clicking on the icon. If the Registry has the wrong file type associated the **jar** files, then double-clicking **jar** icons will not work. In this case, we recommend running DrJava inside the Windows "Command Prompt" or Power Shell using the same command as shown above for Linux.

Legacy documentation for DrJava is available at https://drjava.sourceforge.net; it is still accurate in most respects. If you have further questions about DrJava, ask them on Piazza.

Testing You are expected to write unit tests for all of your non-trivial methods or functions *and submit your test code as part of your program*. DrJava provides integrated support for JUnit which makes this task easy. IntelliJ supports integrated testing using Junit as well.

Task Your task is to write a parser in Generic Java (Java 8+) that recognizes expressions in the pedagogic functional

language Jam, which will be defined subsequently. The course staff has provided a lexer in Java for this assignment. This lexer returns a larger set of symbols (tokens) than what appears in the Jam Syntax Definition below. These extra symbols anticipate future extensions to Jam in later projects. At this point, your parser should simply reject any program containing one of these extra symbols as ill-formed, just like it would for any other syntax error.

Java Support Library In the provided support files, the parser's input token stream is generated by a **Lexer** object supporting the 0-ary method **readToken()**. The **readToken()** method produces a stream of **Token** objects belonging to various subclasses of the **Token** interface. The **Lexer** class supports two constructors: (i) a zero-ary constructor that converts standard input to a **Lexer** and (ii) a unary constructor **Lexer(String fileName)** that converts the specified file to a **Lexer**. The file **lexer.java** contains the definition of the **Lexer** class and the collection of classes representing tokens (the Java interface type **Token**) and abstract syntax trees (the java interface type **AST**).

The API That Your Program Must Support

The API provided in the starter repository exclusively uses must be in the default (top-level) package. All of the classes that you create to augment those in the starter repository should be in the empty package.

The starter repository contains a folder (directory) called **src** containing of the provided source files. We recommend that

you create a parallel directory named **classes** to hold all of the class files generated by the **javac** compiler. You can easily do this in DrJava by creating a project with **src** as the root of the source folder and **classes** as the build folder.

In the **src** folder, the stub file **Parser.java** includes constructors **Parser(Reader stream)** and **Parser(String filename)** that a create parser to read the specified stream or file. The **Reader** variant of these constructors is very convenient when you are testing your parser from the DrJava read-eval-print loop or from unit test methods. The **String** variant supports parsing Jam programs that reside in files.

The stub **Parser** class contains a public method **AST parse()** with a stubbed out body that returns the abstract syntax tree for the Jam expression in the input stream. It also contains one parse method **private AST parseExp()** with a stubbed out body that is the primary helper method for **parse()** along with a contract specifying what **parseExp** does. grading program will print the output using the method **System.out.println**, which invokes the **toString** method on its argument. All of the requisite abstract syntax classes including **toString** methods are defined in the file **AST.java** in the starter repository. All of these support files are located in the folder **src** in the starter repository for Assignment 1.

If your parser encounters an erroneous input (including illegal tokens), simply throw a **ParseException** containing a message explanation explaining the error. This operation will abort parsing the program. The parser in this assignment does

not attempt to recover from syntax errors to continue parsing. This behavior requires a complex specification including many special cases, which is beyond the scope of this assignment.

The starter repository should compile without any errors but the unit test suite in Assign1Test.java will fail since it tries to test unwritten code. This test suite is grossly incomplete; it includes a few sample tests as examples to show you how should write the remaining tests. One measure of the thoroughness of your test suite is how well it covers the statements and branches in your program. Such coverage will almost certainly be incomplete (particularly the branches) since some points in the code may be unreachable and some branches may be impossible to traverse.

Jam Syntax Definition

The following paragraphs define the parser's input language and the subset that your parser should recognize as legal Jam programs.

The Meta-Language

The parser's input language is described using Extended Backus-Naur Form (EBNF). When reading the notation, keep the following in mind:

- A symbol in the meta-language always starts with a capital letter. Therefore, **Symbol** is a symbol in the-meta language, whereas **literal** is the actual text "literal". Note that this convention depends on the fact that no literals in

Jam begin with a capital letter.

- A phrase ***P*** enclosed in braces **{ *P* }** is optional, *i.e*. both ***P*** and the empty string match **{ *P* }**.
- A phrase ***P*** followed by ***** is iterated zero or more times, *e.g*. **Def*** means the concatenation of zero or more **Def** phrases:
  Def
  Def Def
  ...
  and so on, including the empty string.
- A phrase ***P*** followed by **+** is iterated one or more times, *e.g*. **Def+** means the concatenation of one or more **Def** phrases:
  Def
  Def Def
  ...
  and so on, but **not** the empty string.
- If a phrase ***P*** enclosed in brackets **{ *P* }** is followed by a ***** or **+**, the braces are simply denote grouping: the enclosed phrase is iterated as specified by the ***** or **+** symbol. For example, **{ A B }*** is matched by zero or more concatenations of **A B**:

A B

A B A B

and so on, as well as the empty string, but **not A** or **A B A**.

- If something that is interpreted as part of the meta language should be interpreted literally as part of the

language we are describing, it will be enclosed in *single* quotes. Therefore, **'A'** is the actual letter "**A**", not the symbol **A** in the meta language, and ***P*** **'*'** is the phrase ***P*** followed by an actual asterisk, and does not mean that the phase ***P*** should be repeated zero or more times, as described above.

The Input Language

The parser's input language (modulo a few changes described in the section (**Left-associative Binary Operators**) is **Input** where:

Input **::=** *Token**
Token **::=** *AlphaOther AlphaOtherNumeric** **|** *Delimiter* **|** *Operator* **|** *Int*
The lexer recognizes keywords, delimiters (parentheses, commas, semicolons), primitive operations, identifiers, and numbers.

Adjacent tokens must be separated by whitespace (a non-empty sequence of spaces, tabs, and newlines) unless one of the tokens is a delimiter or operator. In identifying operators, the lexer chooses the longest possible match. Hence, **<=** is interpreted as a single token rather than **<** followed by **=**.

Lower **::= a | b | c | d | ... | z**
Upper **::= 'A' | 'B' | 'C' | 'D' | ... | 'Z'**
Other **::= ? | _**
Digit **::= 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9**
Alpha **::=** *Upper* **|** *Lower*
AlphaOther **::=** *Alpha* **|** *Other*

AlphaOtherNumeric **::=** *AlphaOther* **|** *Digit*
Delimiter **::= (|) | [|] | , | ;**
Operator ::= **'+' | - | ~ | '*' | / | = | != | < | > | <= | >= | & | '|' | :=**
Int **::=** *Digit+*

Note: The terminal symbols **'A'**, **'B'**, **...** are enclosed in single quotation marks because **A**, **B**, **...** would automatically be the names of non-terminal symbols in our notation system. For essentially the same reason, we enclose the symbols **'+'**, **'*'**, and **'|'** in single quotation marks; these symbols without quotation marks are metasymbols in the notation scheme described above.

Jam The set of legitimate Jam phrases is the subset of the potential inputs consisting the expressions *Exp* defined by the following grammar:

Expressions

Exp **::=** *Term* { *Binop Exp* }
| **if** *Exp* **then** *Exp* **else** *Exp*
| **let** *Def+* **in** *Exp*
| **map** *IdList* **to** *Exp*
Term **::=** *Unop Term*
| *Factor* { (*ExpList*) }
| *Empty*
| *Int*
| *Bool*
Factor **::=** (*Exp*) **|** *Prim* **|** *Id*
ExpList **::=** { *PropExpList* }

PropExpList **::=** *Exp* { **,** *Exp* }*
IdList **::=** { *PropIdList* }
PropIdList **::=** *Id* { **,** *Id*}*

Definitions

Def **::=** *Id* **:=** *Exp* **;**

Primitive Constants, Operators, and Operations

Empty **::= empty**
Bool **::= true | false**
Unop **::=** *Sign* **| ~**
Sign **::= '+' | -**
Binop **::=** *Sign* **| '*' | / | = | != | < | > | <= | >= | & | '|'**
Prim **::= number? | function? | list? | empty? | cons? | cons | first | rest | arity**

Identifiers

Id **::=** *AlphaOther* {*AlphaOther* **|** *Digit*}*
Please note that *Id* does **not** contain the anything that matches symbols in *Prim* or the keywords **if**, **then**, **else**, **map**, **to**, **let**, **in**, **empty**, **true**, and **false**.

Numbers

Int ::= Digit+
The preceding grammar requires one symbol look-ahead in a few situations. The Java lexer in our support code includes a method **Token peek()** that behaves exactly like the method **Token readToken()** except for the fact that it leaves the scanned token at the front of the input stream (in contrast **readToken()** removes the scanned token from the input

stream).

The Java file **lexer.java** defines all of the abstract syntax classes required to represent Jam programs using the composite design pattern. There is one constructor for each fundamentally different form of expression in the definition of Jam syntax given above. Some primitive (non-recursive) abstract syntax classes are the same the corresponding token classes.

For example, suppose the Jam program under consideration is

f(x) + (x * 12)

The abstract syntax representation for this program phrase is:

```
new BinOpApp(
    new Op("+"),
    new App(new Variable("f"), new AST[] { new
Variable("x") }),
    new BinOpApp(new Op("*"), new Variable("x"), new
IntConstant(12))
  )
```

This construction does not tell the whole story. The lexer is guaranteed to always return the *same* object for a given operator. Similarly, there is only copy of each variable encountered by the lexer. Hence, there is only one copy of **Op("+")**, one copy of the **Op("*")**, one copy of the **Variable("x")**, *etc*. As a result, the **==** operator in Java can be used to compare operators and variables. Note that the operators **Op("+")** and **Op("-")** can be used either as unary

or binary operators.

Left-Associative Binary Operators

The preceding grammar for Jam has an annoying flaw. To make the language easy to parse using recursive descent (top-down) parsing, we used right recursion in the definition of arithmetic expressions (**Exp**). (Left recursion forces infinite looping in top-down parsing.) But right recursion yields right-associative **AST**s (where computation must be performed right to left), while the usual mathematical convention in arithmetic expressions is that binary operators are left-associative (where computation must be performed left to right). There is a widely-used trick in writing grammars for top-down parsing using syntax diagrams that avoids this problem. The trick is replace right recursion in a simple production (only one alternative on the right hand side) by iteration: an edge looping back to the input edge of the diagram. We used this trick in generating the syntax diagrams for **ExpList** and **IdList** for our original grammar. Given our original Jam grammar, we can convert the production

Exp ::= Term { Binop Exp } | ...

to

Exp ::= BinaryExp | ...
BinaryExp ::= Term { Binop BinaryExp }

at the cost of excluding chains of binary operations terminating in an expression other than a term. Then this revised grammar can be represented using syntax diagrams that define **BinaryExp** iteratively (just like **PropExpList** in

the original grammar).

The preceding definition of **BinaryExp** generates the same set of strings as the extended rule:

BinaryExp ::= Term { BinOp Term }*
which is ambiguous about left or right associativity. When iteration is used to recognize a **BinaryExp**, the parsing process can construct a left-associative tree since the tree is constructed left-to-right (each new **Binop** adds a new root to the **AST**). This convention make the syntax diagram formulation of our Jam grammar deterministic and unambiguous.

The file JamSyntaxDiagrams file in Assignment 1 folder contains iterative syntax diagrams corresponding to the original left associative version of the Jam language. These diagrams precisely describe the syntax of Jam and how we want the parser to behave. Of course, this language eliminates a few quirky inputs from the language based on our original grammar. Your parser should treat these quirky inputs as erroneous.

It is possible to write syntax diagrams that treat binary expressions iteratively and still allow arbitrary expressions at the end of the binary expression. But the corresponding grammar and syntax diagrams are much more complex. So we have not followed this design choice in creating the revised syntax for Jam. In fact, we view our Jam grammar as a case study showing that pure context-free grammars are not quite the right formalism for expressing easily parsed program

syntax. Syntax diagrams where iteration is always left-associative (equivalent to extended context-free-grammars where iteration always expands into left-associative parse trees) are much better. I am not aware of a rigorous exposition of such extended BNF grammars in the literature; I suspect that most theoretical computer scientists would not find such an exposition very interesting even though the standard notion of parsing language specified by a context-free grammar is an imperfect formalization of practical parsing.

Write your parser to produce left-associative ASTs for arithmetic expressions. If your parser implements either of the "revised" syntax diagram definitions given above (which are equivalent) using **while** loops for iteration and constructs the corresponding abstract syntax trees in a functional style (no mutation of trees in the construction process), your parser will naturally build syntax trees that are left-associative.

Testing and Submitting Your Program

The src folder in the provided code base includes some Junit4 test classes which are not comprehensive. You need to add more test cases; you need to check every production in the grammar. If a production involves iteration or recursion, you need to test the base cases as well as simple inductive constructions.

Edit the **README** file for your repository so that it

- outlines the organization of your program,
- specifies any compatible additions that you made the provided API (not generally necessary), and

- specifies what testing process you used to confirm the correctness of your program.

In a GitHub repository, the README file is a GitHub "MarkDown" file, text augmented by a few simple formatting annotations. You may use this notation if you wish, but a plain text file is satisfactory as well. GitHub MarkDown (MD) is much less powerful than HTML but much easier to use. The webpage **markdown-cheat-sheet** is a good summary for learning MarkDown and a useful reference.

We do not recommend creating a separate test data file for each class in your Parser program because it creates more opportunities for incidental errors when testing your solution. Unless you are testing how well your program scales to large inputs, such testing is typically unnecessary. All JUnit test classes submitted for grading should be compatible with a generic solution that conforms to the public interface provided by the stub files in the template repository. In other words, we should be able to compile and run your unit tests against a reference solution used for grading, which has the same public interface as the support code. Your test classes should be defined in files matching the pattern "***Assign#Test.java**", where "**#**" is the current assignment number. For example, "**MyAssign2Test.java**" would be a valid name for Assignment 2 unit tests. If you want to include other tests that you do not want graded (*e.g.*, to test your private interfaces), then simply use a different suffix than "**Test**" for the test class name (e.g., "**Assign1Test#.java**"). If you have additional

utility classes that are required dependences of your unit test classes, please name those classes using the pattern **"*Assign#TestSupport.java"**, which signals that the class is required to support your unit tests, but it is not a unit test class that should be independently launched. You may also include test support files with names matching the pattern **"*AssignAllTestSupport.java"**, which can contain common test support code that can be reused across multiple assignment submissions without needing to rename the file. Note that our class solutions to the assignments do not rely on such utility classes.

Each method in your program should include a short comment stating precisely what it does. For routines that parse a particular form of program phrase, the comment should provide the grammar rules describing that form.

To "submit" your program, convert your repository to a **zip** file (a capability built-in to Windows) and upload it using the Canvas assignment interface.

Implementation Hints

The **toString()** methods for each AST class effectively provide an unparser for ASTs. You can directly compare test input strings with output **AST**s by unparsing the **AST**s (using **toString()**). The two strings must match exactly; differences in whitespace, new lines, and parentheses used for grouping may force you to slightly reformat your input strings to match the conventions embedded in the implementation of **toString()**.